

# Week 1: Python basics

Python syntax and code patterns. Keep it next to you while you work.

## Running cells

A variable exists only once its cell has run. **Run All** runs them in order from the top, **Restart** discards every variable, **Clear All Outputs** keeps your code and removes the results. When a notebook misbehaves: Restart, then Run All.

## Variables and types

```
n_trials = 240          # int
rt = 0.42               # float
cohort = "CSHL-WT"      # str
correct = True          # bool
missing = None          # NoneType

type(rt)                # <class 'float'>
int("7"), float("0.5"), str(7)  # convert
```

## Arithmetic

```
+ - * /                / always gives a float
//                     integer division: 7 // 2 is 3
%                      remainder: 7 % 2 is 1
**                     power: 2 ** 3 is 8
x += 1                 shorthand for x = x + 1
```

## print and f-strings

```
print("value =", rt, type(rt))    # commas add spaces
print(n, end=" ")                 # no new line

f"mean RT {rt:.3f} s"            # 0.420
f"accuracy {acc:.1%}"            # 82.0%
f"{n_trials:,.0f} trials"        # 694,182
f"contrast {c:+.4f}"             # +0.2500
f"{lab:<15} {group:<4} {n}"       # pad left, columns line up
f"{n:>5}"                        # pad right
f"p = {p:.4g}"                  # 4 significant digits
```

## Text

```
name = "CSHL" + "_051"          # joining text with +
len(name)                       # how many characters
'single' and "double"           # both work, stay consistent
```

## Lists

```
rts = [0.31, 0.42, 0.28]
rts[0], rts[-1]                 # first, last
rts[0:2]                        # from 0, up to but NOT 2
rts.append(0.55)                # add at the end
rts.sort(); rts.reverse()       # in place, returns nothing
last = rts.pop()                # remove and hand back
len(rts), sum(rts), max(rts), min(rts)
sorted(rts)                     # a sorted copy
0.42 in rts                     # True
```

## Dictionaries

```
mouse = {"id": "CSHL051", "lab": "CSHL"}
mouse["id"]                     # look up by key
mouse["age"] = 14               # add or overwrite
"lab" in mouse                  # True
mouse.keys(), mouse.values()

for key, value in mouse.items():
    print(key, value)
```

## Tuples

```
point = (0.25, 0.83)
x, y = point                    # unpacking
key = (lab, group)              # usable as a dict key
```

A tuple cannot be changed after it is built, which is why it can be a key and a list cannot.

## Conditionals

```
if signed_contrast > 0:
    side = "right"
elif signed_contrast < 0:
    side = "left"
else:
    side = "zero contrast"

== !=                            equal, not equal
< > <= >=                       comparisons
and or not                       combine conditions
is True, is None                 identity, used for True and None
```

## Loops

```
for rt in rts:                  # each item
    print(rt)

for i in range(5):              # 0 1 2 3 4
for n in range(0, 21, 2):       # start, stop, step
for _ in range(100):            # repeat, index unused
```

```
for i, rt in enumerate(rts):    # index and item
for lab, grp in zip(labs, groups): # two lists at once

while trials_left > 0:
    trials_left -= 1

break        # leave the loop now
continue    # skip to the next item
```

## Pattern: count into a dictionary

```
counts = {}
for trial in trials:
    mouse = trial["mouse"]
    if mouse not in counts:
        counts[mouse] = 0
    counts[mouse] += 1

for mouse, n in counts.items():
    print(f"{mouse}: {n} trials")
```

## Pattern: a column as a plain list

```
labs = mice["lab"].tolist()     # out of pandas, into Python
for lab in labs:
    print(lab)

mice["lab"].unique().tolist()   # each value once
```

## Pattern: count with conditions

```
n_correct = 0
n_wrong = 0
for trial in trials:
    if trial["mouse"] != "CSHL046":
        continue        # skip the rest of this turn
    if trial["correct"] is True:
        n_correct += 1
    else:
        n_wrong += 1

accuracy = n_correct / (n_correct + n_wrong)
```

## Reading an error

Read the **last** line first: it names the problem.

|                  |                                         |
|------------------|-----------------------------------------|
| NameError        | a name never defined, often a typo      |
| TypeError        | an operation between incompatible types |
| IndexError       | a position that does not exist          |
| KeyError         | a key or column that is not there       |
| SyntaxError      | Python cannot read the line at all      |
| IndentationError | a block indented inconsistently         |

## Imports and help

```
help(round)        # the full documentation
round?            # a quick summary, in Jupyter

import math, random
import numpy as np
from pathlib import Path
from src.qmn_utils import sigmoid

math.sqrt(50), math.log(1000)
random.seed(1); random.choice(rts)
Path.cwd(), Path("data") / "file.csv"
```

## List comprehensions

```
squares = [n ** 2 for n in range(10)]
evens = [n ** 2 for n in range(10) if n % 2 == 0]
```

A shorter way to write a loop that builds a list. You never have to write one in this course, but they appear in code you read from Week 2 on.

## Files

```
with open("summary.txt", "w") as f:
    for key, value in summary.items():
        f.write(f"{key}: {value}\n")

with open("summary.txt", "r") as f:
    text = f.read()
```

"w" overwrites, "a" appends, "r" reads. The file closes itself.

## A first look at pandas

```
mice = pd.read_csv(path)
mice.head(), mice.shape, mice.info()
mice["lab"]                # one column
mice["lab"].tolist()       # as a plain list
mice["age"].mean()
mice.iloc[0]               # by position
mice.loc[0, "lab"]         # by label
```